学　号：　w20232008

桂林理工大学

GUILIN UNIVERSITY OF TECHNOLOGY

淘宝公司大数据运营环境的搭建和部署

实验报告

学　　院：计算机科学与工程学院

班　　级：计科 23-1

姓　　名：苏蕾宁

指导教师：郭强

评　分/价：

2026 年 6 月 4 日

目录

摘要 3
1. 大数据环境构建 3
2. Linux 系统环境配置 7
3. 大数据相关开发软件安装 13
4. Hadoop 分布式集群环境搭建 17
5. Hive 数据仓库配置 22
6. Flume 数据采集系统 24
7. ZooKeeper 集群环境搭建 25
8. Kafka 集群环境搭建 27

9. 数据仓库分层设计 28
10. 数据可视化 38
- 10.1 环境准备 37
- 10.2 数据导出到 MySQL 37
- 10.3 Python 数据可视化脚本 Pycharts.....41

摘要:

本报告以淘宝公司为背景，系统阐述了电商大数据运营环境的完整搭建与部署方案。淘宝作为中国最大的电商平台，拥有超过 8 亿活跃用户，每日产生 TB 级用户行为日志和百万级交易订单数据，对大数据处理能力提出了极高要求。

本方案基于 Hadoop 生态技术栈，从 Linux 环境配置、Hadoop 集群搭建、HDFS 分布式存储、YARN 资源调度、Hive 数据仓库建设、Flume 日志采集、Sqoop 数据迁移、ZooKeeper 协调服务、Kafka 消息队列到数据可视化的全流程进行详细设计。通过构建 ODS→DWD→DWS→ADS 四层数仓架构，实现用户行为分析、活跃用户统计、留存率计算、用户画像、营销效果分析等核心业务指标，最终通过 pyecharts 实现数据大屏可视化展示。

本报告使用的数据集来源于公开数据集，包含 6019 条用户行为记录，涵盖用户人口统计信息、浏览行为、设备类型、广告点击、购买转化等 14 个字段，完整呈现了从数据采集到业务应用的大数据全链路实践。

关键词： 淘宝电商；大数据平台；Hadoop；HDFS；YARN；Hive；Flume；Kafka；数据可视化

参考原来数据： https://www.kaggle.com/datasets/okiasstephanie/e-commerce-user-behaviour-data

1 大数据环境构建

1.1 项目背景

1.1.1 淘宝业务概况

淘宝（Taobao）是中国最大的电商平台，由阿里巴巴集团于 2003 年创立。经过二十余年的发展，淘宝已从最初的 C2C 交易平台发展成为涵盖 B2C、C2C、直播电商、本地生活等多种业态的综合电商平台。

淘宝平台的核心业务特征包括：

- 用户规模庞大：年度活跃消费者超过 8 亿
- 商品种类丰富：覆盖服装、数码、家居、美妆、食品等全品类
- 交易高频：每日产生数千万笔订单
- 数据维度多样：用户行为日志、交易订单、商品信息、物流状态、评价反馈等

1.1.2 大数据建设的必要性与挑战

随着淘宝业务规模的持续扩张，数据量呈指数级增长，传统的数据处理架构面临严峻挑战：

（1）数据体量激增

淘宝平台每日产生海量数据：

- 用户行为日志：每日 TB 级别
- 交易订单：每日千万级
- 商品信息：亿级商品 SKU
- 用户画像：亿级用户画像标签

（2）实时性要求提升

- 大促活动（双十一）需要实时监控 GMV、订单量等核心指标
- 个性化推荐需要基于用户最新行为实时更新
- 风险控制系统需要秒级识别异常交易

（3）数据来源多样化

- Web 端用户行为日志
- APP 端用户点击流埋点
- MySQL 业务数据库交易记录
- 服务器访问日志

（4）数据质量与治理需求

- 确保数据的准确性、一致性、完整性
- 建立统一的数据标准和指标体系
- 实现数据的安全合规使用

1.2 建设目标

本大数据平台建设项目的总体目标是构建一个稳定、高效、可扩展、易治理的企业级大数据运营环境。

1.2.1 技术目标

1.2.2业务目标

目标维度	具体指标	说明
数据采集	支持 TB 级日志采集，采集成功率≥99.9%	支持多端数据采集
数据同步	支持异构数据源同步，延迟≤5 分钟	MySQL 到 Hive 实时同步
离线计算	支持 PB 级数据处理，任务完成率≥99.99%	每日凌晨完成 T-1 数据产出
实时计算	端到端延迟≤1 分钟	支撑实时大屏
数据服务	API 响应时间≤100ms	支持高并发数据查询
系统可用性	全年可用性≥99.99%	7×24 小时稳定运行

- 构建统一的数据中台，实现数据资产的共享与复用
- 建立完善的数据治理体系，确保数据质量与安全
- 支撑实时运营决策，提升业务响应速度
- 赋能用户画像与推荐系统，驱动智能化转型
- 降低数据研发与运维成本，提升人效

1.2.3 数据规模预估

基于淘宝业务规模预估：

数据类型	日增量	保留周期	总存储量
用户行为日志	500GB-1TB	30 天	15-30TB
交易订单数据	100GB	90 天	9TB
用户画像数据	50GB	365 天	18TB
商品数据	20GB	365 天	7.3TB
汇总报表数据	10GB	永久	持续累积

1.3 设计原则

本方案遵循以下设计原则：

（1）分层解耦原则

将数据平台划分为采集层、存储层、计算层、服务层、应用层，各层之间通过标准化接口交互，降低耦合度，便于独立扩展和维护。

（2）存储计算分离原则

采用存储与计算分离的架构设计，存储层统一使用 HDFS 分布式文件系统，计算层按需弹性伸缩。

（3）数据复用原则

遵循数据仓库分层设计，ODS→DWD→DWS→ADS，下层数据为上层复用，避免重复计算。

（4）安全可控原则

建立完善的数据权限管控、数据脱敏、审计日志机制，确保数据使用合规安全。

（5）成本可控原则

通过数据生命周期管理、资源优化等手段，实现数据平台的精细化成本管控。

1.4 数据源说明

本项目使用的数据集来源于公开数据集，包含 6019 条用户行为记录。数据字段说明如下：

字段名	类型	说明	示例
user_id	int	用户唯一标识	1
age	string	年龄段	25-34
gender	string	性别	Male/Female
device_type	string	设备类型	Mobile/Desktop/Tablet
time_on_site	int	网站停留时长(秒)	354
pages_viewed	int	浏览页面数	12
previous_purchases	int	历史购买次数	3
cart_item	int	加购商品数	2
discount_seen	int	是否看到促销(0/1)	1
ad_clicked	int	是否点击广告(0/1)	1
returning_user	int	是否回访用户(0/1)	1
avg_session_time	int	平均会话时长(秒)	118
bounce_rate	float	跳出率(0-1)	0.25
purchase	int	是否购买(0/1)	1

2.Linux 系统环境配置

2.1 集群规划

淘宝大数据平台采用 3 节点集群架构：

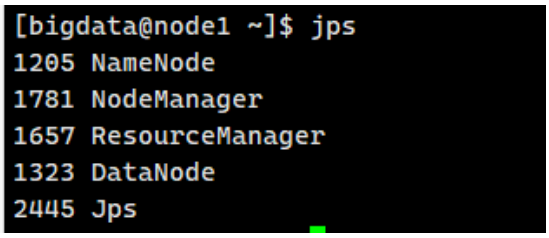
节点名称	IP 地址	角色	部署组件
node1	192.168.100.3	主节点	NameNode, ResourceManager, Hive, MySQL, ZooKeeper
node2	192.168.100.4	从节点	DataNode, NodeManager, ZooKeeper, Kafka
node3	192.168.100.5	从节点+辅助	SecondaryNameNode, DataNode, NodeManager, ZooKeeper, Kafka, Flume

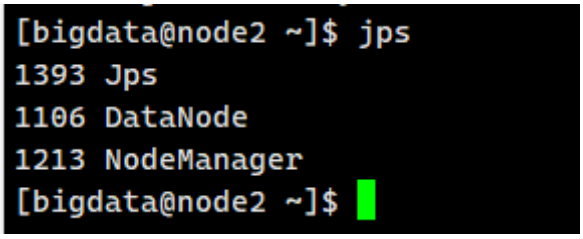
2.2 HDFS 集群服务规划

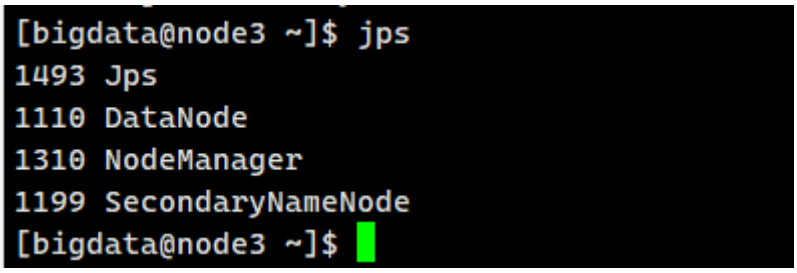
在分布式集群中，每一个节点都会部署多个服务，所以需要对三个节点进行 HDFS 集群服务进行规划。

配置完毕，启动 HDFS 集群后：

- node1 节点:运行 NameNode、DataNode、ResourceManager,NodeManager 共 4 个服务
- node2 节点：运行 DataNode、NodeManager 共 2 个服务
- node3 节点：运行 SecondaryNameNode、DataNode、NodeManager 共 3 个服务







2.3 虚拟机环境准备

本实验以 node1 虚拟机为克隆源，通过克隆方式完成 node2、node3 虚拟机的安装，形成 node1、node2、node3 三台虚拟机构成的分布式集群。

2.3.1 IP 地址规划

为了搭建分布式集群，需要对整个集群中每个节点的 IP 地址进行统一规划：

节点名称	IP 地址	子网掩码	网关
node1	192.168.100.3	255.255.255.0	192.168.100.2
node2	192.168.100.4	255.255.255.0	192.168.100.2
node3	192.168.100.5	255.255.255.0	192.168.100.2

2.3.2 虚拟机网络配置

（1）在 node1 节点打开网络环境配置文件 ifcfg-ens33.

```
[root@node1 ~]# vi /etc/sysconfig/network-scripts/ifcfg-ens33
```

(2)按键盘“a”或者“i”进入编辑模式，配置 node1 节点的 IP 地址、网关、子网掩码、DNS 解析器。如图所示，输入或修改如下内容，保存并退出.

```
BOOTPROTO=static

IPADDR=192.168.100.3

GATEWAY=192.168.100.2

NETMASK=255.255.255.0

DNS1=114.114.114.114

DNS2=8.8.8.8
```

(3）重启 node1 节点的网络环境，使网络环境生效。

```
[root@node1 ~]# service network restart
```

node1 虚拟机为克隆源，通过克隆方式完成 node2、node3 虚拟机的安装，形成 node1、node2、node3 三台虚拟机构成的分布式集群。为了搭建分布式集群，需要对整个集群中每个节单击的 IP 地址进行 IP 规划。因此在本集群中，规划 node1 节点配置 IP 地址为 192.168.100.3，node2 节点配置 IP 地址为 192.168.100.4，node3 节点配置 IP 地址为 192.168.100.5。

- 修改 hosts 文件

（1）修改 node1 节点的 hosts 文件在 node1 虚拟机上打开 hosts 文件。

```
[root@node1 ~]# vi /etc/hosts
```

在 hosts 文件内容末尾添加三台虚拟机 IP 地址及对应的主机名，内容如下

```
192.168.100.3 node1
192.168.100.4 node2
192.168.100.5 node3
```

(2）修改 node2、node3 节点的 hosts 文件
分别在 node2、node3 虚拟机上修改 hosts 文件，操作步骤同上。

2.4 创建普通用户

分布式集群中的每一个节点都要创建普通用户。在开发过程中，无法使用超级用户 root 来对系统进行操作，需要 root 用户给我们开发人员创建一个普通用户进行后期的操作系统使用。因此需要在 node1、node2、node3 虚拟机上面创建一个普通用户，后期所有的开发操作都使用这个普通用户来完成。
(1）打开 Xshell 软件，单击菜单栏中的“查看-撰写”，勾选“撰写栏”，在底部单击 图标选择撰写栏中“全部会话”；
（2）在撰写栏中输入命令 useradd bigdata，回车后 node1、node2、node3 虚拟 机中会创建 bigdata 用户。
（3）设置 node1、node2、node3 三个节点 bigdata 用户的密码，在撰写栏内输入 passwd bigdata 命令，回车后在撰写栏中输入密码 123456，提示重新输入新的密码后 再次输入密码 123456。

2.5 SSH 免密登录配置

SSH 免密登录是分布式集群正常运行的前提，各节点之间需要能够无密码相互访问。

因为在整个分布式集群中的每一个节点上，部署很多大数据的相关服务，服务与 服务在访问的过程中会有密码校验这样一个安全机制。为了避免频繁的输入密码，需 要对分布式集群中的每一个节点进行免密登录，所以需要配置免密登录。 1）bigdata 用户的免密登录配置
（1)切换为普通用户的登录模式 在撰写栏中输入命令 su?bigdata,回车,就会将 node1、node2、node3 三个节点切换为普通用户的登录模式.

（2） 产生公钥和私钥对 在撰写栏中输入命令 ssh-keygen -t rsa，然后回车四次，产生公钥和私钥。
（3） （3)复制公钥到每一台虚拟机 A. 在撰写栏中再输入命令:ssh-copy-id node1 ，将公钥复制到 node1 节点上，回车.

2.6 防火墙与 SELinux 配置

防火墙相关操作 首先，关闭 node1 节点的防火墙。

```
[root@node1 ~]# systemctl stop firewalld.service
```

然后，禁用 node1 节点的防火墙。

```
[root@node1 ~]# systemctl disable firewalld.service
```

最后，通过命令查看防火墙的状态

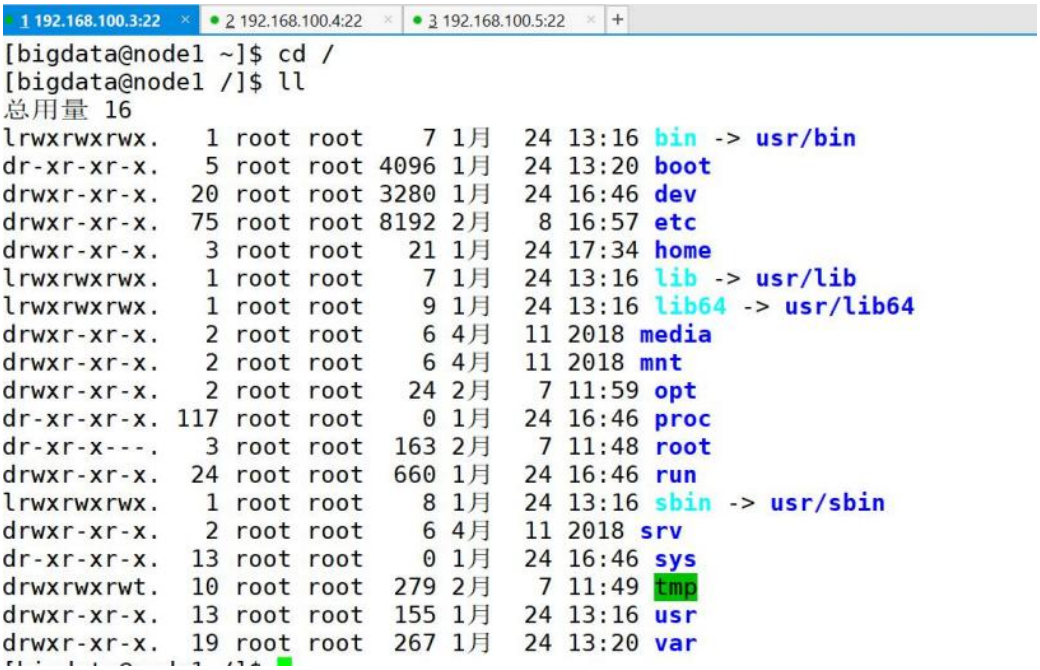
```
[root@node1 ~]# systemctl status firewalld.service
```

执行结果中出现“dead”，说明防火墙关闭成功.

2.7 Linux 系统目录结构详解

2.7.1 Linux 根目录结构

使用命令 cd /进入到 Linux 的根目录下，使用命令 ll 查看根目录下的所有内容，Linux 系统的目录结构如下图所示：



2.7.2 重要目录详解

（1）etc 目录

etc 目录用来存放系统管理所需要的配置文件，系统所有的配置文件全部存储在该目录下。在后续的大数据平台搭建过程中，需要修改的 profile、hosts、hostname 等文件都位于此目录下。

查看 etc 目录下的配置文件

```
ls /etc/ | grep -E "profile|hosts|hostname"
```

（2）home 目录

home 目录存放普通用户的根目录，初始为空。在 Linux 系统中每个用户都有一个自己的家目录，比如我们创建了普通用户 bigdata，那么在 Linux 的 home 目录中，就存在/home/bigdata 目录。

查看 home 目录

```
ls -la /home/
```

（3）opt 目录

opt 目录是给主机额外安装软件所设置的目录，默认为空。该目录是后续在开发项目过程中使用最多的目录，在该目录下存放着项目所安装的全部软件及所有数据文件。这是本课程大数据平台搭建的核心目录。

2.8 项目路径规划

在后续项目开发过程中，需要对 Linux 系统路径进行统一规划。所有开发操作都集中在/opt 目录下进行。

2.8.1 opt 目录结构规划

在/opt 目录下创建四个子目录，分别是 module、project、software 和 testData。

```
# 创建 opt 目录下的子目录
sudo mkdir -p /opt/module
sudo mkdir -p /opt/project
sudo mkdir -p /opt/software
sudo mkdir -p /opt/testData

# 修改目录所有者
sudo chown -R bigdata:bigdata /opt/*
```

各目录功能说明：

目录	用途	存储内容
/opt/software	存放各种软件安装包	Hadoop、Flume、Kafka、Hive、Sqoop、MySQL、Zookeeper、JDK 等安装包
/opt/module	软件的安装目录	Hadoop、Flume、Kafka、Hive、Sqoop、Zookeeper、JDK 等解压后的程序
/opt/project	项目操作目录	项目开发过程中产生的所有文件
/opt/testData	测试数据目录	测试脚本文件或数据文件

2.8.2 project 目录详细规划

在/opt/project 目录下创建 offlineDataWarehouse 目录，该目录是本项目的操作目录。在该目录下,再创建三个子目录,分别是 data_collection、jar、logs 和 user_behavior_logs。

bash

```
# 创建项目目录结构
mkdir -p /opt/project/offlineDataWarehouse/data_collection
mkdir -p /opt/project/offlineDataWarehouse/jar
mkdir -p /opt/project/offlineDataWarehouse/logs
mkdir -p /opt/project/offlineDataWarehouse/user_behavior_logs

# 查看创建的目录结构
tree /opt/project/
```

各子目录功能说明：

目录	路径	用途
data_collection	/opt/project/offlineDataWarehouse/data_collection	存放Flume数据采集系统相关脚本文件
jar	/opt/project/offlineDataWarehouse/jar	存放生成数据源的jar包和运行脚本
logs	/opt/project/offlineDataWarehouse/logs	存放项目运行过程中产生的所有日志文件
user_behavior_logs	/opt/project/offlineDataWarehouse/user_behavior_logs	存放模拟生成的用户行为数据

2.8.3 完整项目路径规划结构图

当项目完成之后，目录的总体结构和其中的主要内容如下图所示：

Node1

```
/opt/
├─── software/                          # 软件安装包存放目录
│    ├─── apache-hive-3.1.2-bin.tar.gz
│    ├─── apache-zookeeper-3.5.6-bin.tar.gz
│    ├─── hadoop-3.3.0.tar.gz
│    ├─── jdk-8u251-linux-x64.tar.gz
│    ├─── kafka_2.12-2.7.0.tgz
│    ├─── mysql-5.7.28-1.el7.x86_64.rpm-bundle.tar
│    ├─── sqoop-1.4.7.bin_hadoop-2.6.0.tar.gz
│    └─── flume-1.9.0.tar.gz
│
├─── module/                            # 软件安装目录
│    ├─── hadoop-3.3.0/                 # Hadoop 安装目录
│    ├─── hive-3.1.2/                   # Hive 安装目录
│    ├─── jdk1.8.0_251/                 # JDK 安装目录
│    ├─── kafka_2.12-2.5.0/             # Kafka 安装目录
│    ├─── sqoop-1.4.7/                  # Sqoop 安装目录
│    └─── zookeeper-3.5.6/              # ZooKeeper 安装目录
│
├─── project/                           # 项目目录
│    └─── offlineDataWarehouse/         # 离线数据仓库项目
│         ├─── data_collection/         # 数据采集脚本
│         ├─── jar/                     # Jar 包存放
│         ├─── logs/                    # 日志文件
│         └─── user_behavior_logs/      # 用户行为日志数据
```

Node2

```
├──/opt /project/                          # 项目目录
|    └── offlineDataWarehouse/          # 离线数据仓库项目
|        ├── data_collection/           # 数据采集脚本
```

3 大数据相关开发软件安装

3.1 软件安装概述

淘宝大数据平台需要安装以下核心软件组件，形成完整的技术栈体系：

软件名称	版本	功能描述	部署节点
CentOS	7.9	基础运行环境	所有节点
JDK	1.8.0_251	Java 运行环境	所有节点
Hadoop	3.3.0	分布式存储与计算	集群所有节点
Hive	3.1.2	数据仓库工具	node1 节点
Flume	1.9.0	日志采集	node3 节点
Sqoop	1.4.7	数据迁移工具	node1 节点
ZooKeeper	3.5.6	分布式协调服务	所有节点
Kafka	2.12-2.7.0	分布式消息队列	node2、node3 节点
MySQL	5.7.28	Hive 元数据库	node1 节点

3.2 软件安装包准备

所有软件安装包存放在/opt/software 目录下：

bash

查看软件安装包

```
[bigdata@node1 ~]$ ls /opt/software
apache-flume-1.9.0-bin.tar.gz                  mysql-community-devel-5.7.28-1.el7.x86_64.rpm
apache-hive-3.1.2-bin.tar.gz                   mysql-community-embedded-5.7.28-1.el7.x86_64.rpm
apache-zookeeper-3.5.6-bin.tar.gz              mysql-community-embedded-compat-5.7.28-1.el7.x86_64.rpm
commons-lang-2.6-bin.tar.gz                    mysql-community-embedded-devel-5.7.28-1.el7.x86_64.rpm
hadoop-3.3.0.tar.gz                            mysql-community-libs-5.7.28-1.el7.x86_64.rpm
jdk-8u251-linux-x64.tar.gz                     mysql-community-libs-compat-5.7.28-1.el7.x86_64.rpm
kafka_2.12-2.7.0.tgz                           mysql-community-server-5.7.28-1.el7.x86_64.rpm
mysql-5.7.28-1.el7.x86_64.rpm-bundle.tar       mysql-community-test-5.7.28-1.el7.x86_64.rpm
mysql-community-client-5.7.28-1.el7.x86_64.rpm sqoop-1.4.7.bin__hadoop-2.6.0.tar.gz
mysql-community-common-5.7.28-1.el7.x86_64.rpm
```

3.3 JDK 安装配置

JDK 是运行所有 Java 大数据组件的基础环境，必须在集群每个节点上安装。

bash

1. 解压 JDK 到 module 目录

```
[bigdata@node1 /]$ cd /opt/
```

2. 配置环境变量

```
sudo vim /etc/profile
```

```
export JAVA_HOME=/opt/module/jdk1.8.0_251
export PATH=$PATH:$JAVA_HOME/bin
```

3. 使环境变量生效

```
source /etc/profile.d/java.sh
```

4. 验证安装

```
java -version
```

```
[bigdata@node1 jdk1.8.0_251]$ java -version
java version "1.8.0_251"
Java(TM) SE Runtime Environment (build 1.8.0_251-b08)
Java HotSpot(TM) 64-Bit Server VM (build 25.251-b08, mixed mode)
[bigdata@node1 jdk1.8.0_251]$
```

5. 将 JDK 分发到其他节点

```
scp -r /opt/module/jdk node2:/opt/module/
scp -r /opt/module/jdk node3:/opt/module/
scp /etc/profile.d/java.sh node2:/etc/profile.d/
scp /etc/profile.d/java.sh node3:/etc/profile.d/
```

3.4 Hadoop 框架安装

Hadoop 是大数据平台的核心框架，提供 HDFS 分布式存储和 YARN 资源调度能力。

bash

1. 解压 Hadoop 安装包

```
[bigdata@node1 /]$ cd /opt/software/
```

```
[bigdata@node1 software]$ tar -zxvf hadoop-3.3.0.tar.gz -C /opt/module
```

2. 重命名目录

```
mv /opt/module/hadoop-3.3.0 /opt/module/hadoop
```

3. 配置 Hadoop 环境变量

```
export JAVA_HOME=/opt/module/jdk1.8.0_251
export PATH=$PATH:$JAVA_HOME/bin
export HADOOP_HOME=/opt/module/hadoop-3.3.0
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HIVE_HOME=/opt/module/hive-3.1.2
export PATH=$PATH:$HIVE_HOME/bin
export FLUME_HOME=/opt/module/flume-1.9.0
export PATH=$PATH:$FLUME_HOME/bin
export SQOOP_HOME=/opt/module/sqoop-1.4.7
export PATH=$PATH:$SQOOP_HOME/bin
export ZOOKEEPER_HOME=/opt/module/zookeeper-3.5.6
export PATH=$PATH:$ZOOKEEPER_HOME/bin
export KAFKA_HOME=/opt/module/kafka_2.12-2.7.0
export PATH=$PATH:$KAFKA_HOME/bin
"/etc/profile" 90L, 2361C
```

生效系统环境变量配置文件

```
bigdata@node1 hadoop-3.3.0]$ source /etc/profile
```

3.5 Hive 安装

```
bash
# 解压 Hive 安装包
cd /opt/software
tar -zxvf apache-hive-3.1.2-bin.tar.gz -C /opt/module/
mv /opt/module/apache-hive-3.1.2-bin /opt/module/hive

# 配置 Hive 环境变量
cat >> /etc/profile.d/hive.sh << 'EOF'
export HIVE_HOME=/opt/module/hive
export PATH=$PATH:$HIVE_HOME/bin
EOF
source /etc/profile.d/hive.sh
```

3.6 Flume 安装

```
bash
# 解压 Flume 安装包
cd /opt/software
tar -zxvf flume-1.9.0.tar.gz -C /opt/module/
mv /opt/module/flume-1.9.0 /opt/module/flume

# 配置 Flume 环境变量
cat >> /etc/profile.d/flume.sh << 'EOF'
export FLUME_HOME=/opt/module/flume
export PATH=$PATH:$FLUME_HOME/bin
EOF
source /etc/profile.d/flume.sh
```

3.7 Sqoop 安装

```
bash
# 解压 Sqoop 安装包
cd /opt/software
tar -zxvf sqoop-1.4.7.bin_hadoop-2.6.0.tar.gz -C /opt/module/
mv /opt/module/sqoop-1.4.7.bin_hadoop-2.6.0 /opt/module/sqoop

# 配置 Sqoop 环境变量
cat >> /etc/profile.d/sqoop.sh << 'EOF'
export SQOOP_HOME=/opt/module/sqoop
export PATH=$PATH:$SQOOP_HOME/bin
EOF
source /etc/profile.d/sqoop.sh
```

3.8 ZooKeeper 安装

```
bash
# 解压 ZooKeeper 安装包
cd /opt/software
tar -zxvf apache-zookeeper-3.5.6-bin.tar.gz -C /opt/module/
mv /opt/module/apache-zookeeper-3.5.6-bin /opt/module/zookeeper

# 配置 ZooKeeper 环境变量
cat >> /etc/profile.d/zookeeper.sh << 'EOF'
export ZK_HOME=/opt/module/zookeeper
export PATH=$PATH:$ZK_HOME/bin
EOF
source /etc/profile.d/zookeeper.sh
```

3.9 Kafka 安装

```
bash
# 解压 Kafka 安装包
cd /opt/software
tar -zxvf kafka_2.12-2.7.0.tgz -C /opt/module/
mv /opt/module/kafka_2.12-2.7.0 /opt/module/kafka

# 配置 Kafka 环境变量
cat >> /etc/profile.d/kafka.sh << 'EOF'
export KAFKA_HOME=/opt/module/kafka
export PATH=$PATH:$KAFKA_HOME/bin
EOF
source /etc/profile.d/kafka.sh
```

3.10 MySQL 安装

```
bash
# 安装 MySQL
cd /opt/software
tar -xvf mysql-5.7.28-1.el7.x86_64.rpm-bundle.tar
rpm -ivh mysql-community-common-5.7.28-1.el7.x86_64.rpm
rpm -ivh mysql-community-libs-5.7.28-1.el7.x86_64.rpm
rpm -ivh mysql-community-client-5.7.28-1.el7.x86_64.rpm
rpm -ivh mysql-community-server-5.7.28-1.el7.x86_64.rpm

# 启动 MySQL 服务
systemctl start mysqld
systemctl enable mysqld

# 获取临时密码
grep 'temporary password' /var/log/mysqld.log

# 安全配置
mysql_secure_installation

# 创建 Hive 元数据库
mysql -u root -p
```

```
[bigdata@node1 /]$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 2
Server version: 5.7.28 MySQL Community Server (GPL)

Copyright (c) 2000, 2019, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

4. Hadoop 分布式集群环境搭建

4.1 Hadoop 配置文件修改

所有 Hadoop 配置文件位于$HADOOP_HOME/etc/hadoop/目录下。

4.1.1 core-site.xml 配置

双击 core-site.xml 文件，这个文件是 Hadoop 官方提供的配置模板，文件中没有相关配置，需要在这个模板中添加自定义配置内容。将内容添加至<configuration> 与 </configuration>标签之间，配置好后单击保存按钮

```
<configuration>
<property>
<name>fs.defaultFS</name>
<value>hdfs://node1:9000</value>
<description>指定HDFS中NameNode的地址，9000：RPC协议访问端口（远
程调用）</description>
</property>
<property>
<name>hadoop.tmp.dir</name>
<value>/opt/module/hadoop-3.3.0/metaData</value>
<description>指定hadoop运行时产生文件的存储目录，默认路径存在被删除
的可能，要求要么不存在，要么是空路径</description>
</property>
<property>
<name>hadoop.proxyuser.bigdata.hosts</name>
<value>*</value>
<description>hadoop与hive兼容性设置：
Hadoop2.0版本开始支持ProxyUser的机制。含义是使用UserA的用户认
证信息，以UserB的名义去访问hadoop集群。对于服务端来说就认为此时是UserB
在访问集群，相应对访问请求的鉴权（包括HDFS文件系统的权限，YARN提交任务队列的权限）都以用户UserB来进行。UserA被认为是super（这里superuser并不
等同于HDFS中的超级用户，只是拥有代理某些用户的权限，对于HDFS来说本身也是
普通用户），UserB被认为是proxyuser。
</description>
</property>
```

4.1.2 hdfs-site.xml 配置

双击 hdfs-site.xml 文件，这个文件是 Hadoop 官方提供的配置模板，文件中没有相关配置，需要在这个模板中添加自定义配置内容。将内容添加至<configuration> 与 </configuration>标签之间，配置好后单击保存按钮

```
<configuration>

<property>
<name>dfs.replication</name>
<value>3</value>
<description>指定 HDFS 副本的数量</description>
</property><property>
<name>dfs.namenode.name.dir</name>
<value>file://${hadoop.tmp.dir}/name</value>
<description>namenode存储元数据目录</description>
</property>
<property>
<name>dfs.datanode.data.dir</name>
<value>file://${hadoop.tmp.dir}/data</value>
<description>datanode存储数据目录</description>
</property>
<property>
<name>dfs.namenode.checkpoint.dir</name>
<value>file://${hadoop.tmp.dir}/namesecondary</value>
<description>secondary存储合并文件目录</description>
</property>
<property>
<name>dfs.namenode.secondary.http-address</name>
<value>node3:9868</value>
<description>secondaryhttp访问端口</description>
</property>
<property>
<name>dfs.client.datanode-restart.timeout</name>
<value>30</value>
<description>客户端访问datanode超时：30秒</description>
</property>
</configuration>
```

4.1.3 yarn-site.xml 配置

使用 Notepad++软件连接 node1 节点，进入到/opt/module/hadoop-3.3.0/etc/hadoop 目录，双击打开 yarn-site.xml 配置文件，将以下内容添加至<configuration> 与</configuration>标签之间，配置好后单击“保存”图标。

1 node1 | 2 node2 | 3 node3 | +

```
<configuration>

<!-- Site specific YARN configuration properties -->
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
<description>需要被设置 Shuffle 服务的Map Reduce 应用程序</description>
</property>
<property>
<name>yarn.resourcemanager.hostname</name>
<value>node1</value>
<description>resourcemanager服务运行节点</description>
</property>
<property>
<name>yarn.nodemanager.env-whitelist</name>
<value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HAD
OOP_CONF_DIR,CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME,HADOOP_
MAPRED_HOME</value>
<description>环境变量通过从NodeManagers的容器继承的环境属性：
对于mapreduce应用程序，除了默认值hadoopop_mapred_home应该被添加
外。属性值还有JAVA_HOMEHADOOP_COMMON_HOME、HADOOP_HDFS_HOME
HADOOP_CONF_DIR,CLASSPATH_PREPEND_DISTCACHE,HADOOP_YARN_HOME
HADOOP_MAPRED_HOME
</description>
```

4.1.4 mapred-site.xml 配置

双击打开 mapred-site.xml 配置文件，将以下内容添加至<configuration> 与</configuration>标签之间，配置好后单击“保存”图标

```
[bigdata@node1 /]$ cat $HADOOP_HOME/etc/hadoop/mapred-site.xml
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
<property>
<name>mapreduce.jobhistory.address</name>
<value>node1:10020</value>
<description>历史服务地址，RPC协议</description>
</property>
<property>
<name>mapreduce.jobhistory.webapp.address</name>
<value>node1:19888</value>
<description>历史服务地址，http协议</description>
</property>
</configuration>
```

4.1.5 workers 文件配置

```
[bigdata@node1 hadoop]$ cat /opt/module/hadoop-3.3.0/etc/hadoop/workers
node1
node2
node3
[bigdata@node1 hadoop]$
```

4.1.6 hadoop-env.sh 配置

双击 hadoop-env.sh 文件，这个文件是对 JDK 的路径进行修改，在第 54 行的位置，需要将删除“#”号取消注释，再 JAVA_HOME 后面指定 JDK 安装路径。在 node2 窗口中使用 echo $JAVA_HOME 命令，将 JDK 路径输出并复制粘贴至配置文件中，配置好后单击保存按钮。

```
export JAVA_HOME=/opt/module/jdk1.8.0_251
```

4.2 配置文件分发

将 Hadoop 配置文件分发到 node2 和 node3 节点

```
[bigdata@node1 module]$ scp -r hadoop-3.3.0/ bigdata@node2:`pwd`
[bigdata@node1 module]$ scp -r hadoop-3.3.0/ bigdata@node3:`pwd`
```

node1 节点的系统环境变量文件拷贝到 node2、node3 节点。

```
[bigdata@node1 module]$ sudo scp /etc/profile node2:/etc/
[bigdata@node1 module]$ sudo scp /etc/profile node3:/etc/
```

生效 node2、node3 节点的系统环境变量。

```
[bigdata@node2 ~]$ source /etc/profile
[bigdata@node3 ~]$ source /etc/profile
```

4.3 启动 HDFS 集群

现在所有准备工作已经全部做好，在 node1 节点上启动 HDFS 集群。

（1）首先对 HDFS 集群进行格式化，使前面所有配置文件生效，命令可以在任意路径下执行

```
[bigdata@node1 module]$ hdfs namenode -format
```

启动 HDFS 集群，通过 jps 命令查看各个节点进程，如果进程存在，说明 HDFS 集群配置成功

```
[bigdata@node1 hadoop]$ start-dfs.sh
Starting namenodes on [node1]
Starting datanodes
Starting secondary namenodes [node3]
[bigdata@node1 hadoop]$
```

在 node2 节点会存在 DataNode 进程

```
[bigdata@node2 ~]$ jps
2005 Jps
1947 DataNode
[bigdata@node2 ~]$
```

在 node3 节点会存在 SecondaryNameNode 和 DataNode 两个进程

```
[bigdata@node3 ~]$ jps
2528 DataNode
2691 Jps
2614 SecondaryNameNode
[bigdata@node3 ~]$
```

到此，HDFS 集群配置完成并启动成功

4.4 启动 YARN 集群

在 node1 节点命令行中输入启动 YARN 服务的命令

```
[bigdata@node1 hadoop]$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
[bigdata@node1 hadoop]$
```

在 node1、node2、node3 节点的命令行中输入 jps 命令查看 YARN 集群进程

（1）node1 节点在 YARN 服务启动后中会显示 ResourceManager、NodeManager 进程，如下所示

```
[bigdata@node1 hadoop]$ jps
4757 NodeManager
4634 ResourceManager
4123 NameNode
5117 Jps
4238 DataNode
[bigdata@node1 hadoop]$
```

（2）node2 节点在 YARN 服务启动后中会显示 NodeManager 进程，如下所示

```
[bigdata@node2 ~]$ jps
2084 NodeManager
2198 Jps
1947 DataNode
[bigdata@node2 ~]$
```

(3）node3 节点在 YARN 服务启动后中会显示 NodeManager 进程，如下所示：

```
[bigdata@node3 ~]$ jps
2528 DataNode
2882 Jps
2614 SecondaryNameNode
2762 NodeManager
[bigdata@node3 ~]$
```

5. Hive 数据仓库配置

5.1 Hive 安装配置

Hive 是基于 Hadoop 技术的数据仓库工具，用来对数据进行提取、转化和加载，Hive 定义了类似于 SQL 语言的 HQL 语言，便于数据开发和分析人员，完成海量数据的统计和分析。因为 Hive 不是集群架构，所以只需要在一个节点上安装 Hive，并进行配置即可，本项目中，将 Hive 安装在 node1 节点上。

本任务首先对 Hive 安装包进行上传、解压，接下来配置环境变量，最后修改 Hive

- 使用 Xftp 软件将 Hive 安装包 apache-hive-3.1.2-bin.tar.gz 上传至 /opt/software 中

- 将 apache-hive-3.1.2-bin.tar.gz 安装包解压到/opt/module 目录
- 切换到 module 目录，将 apache-hive-3.1.2-bin 文件更名为 hive-3.1.2

1. 双击打开 hive-env.sh 文件，删除 48 行的“#”，设置 HADOOP_HOME 为 Hadoop 的安装路径，删除 51 行的“#”，设置 HIVE_CONF_DIR 为 Hive 配置文件的路径，如下图所示。

```
# Hive Configuration Directory can be controlled by:
 export HIVE_CONF_DIR=/opt/module/hive-3.1.2/conf
```

5.2 Hive 启动与验证

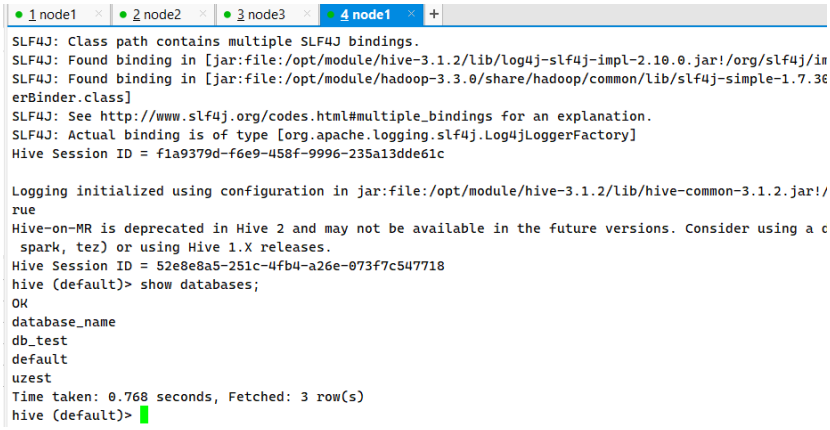
在 node1 命令行中执行如下命令启动元数据服务

```
[bigdata@node1 lib]$ hive --service metastore
2026-05-30 14:18:12: Starting Hive Metastore Server
SLF4J: Class path contains multiple SLF4J bindings.
```

再复制一个 node1 会话窗口，使用 jps 命令查看启动的服务进程，RunJar 进程代表 metastore 服务，如下所示

```
[bigdata@node1 ~]$ jps
4757 NodeManager
5449 Jps
4634 ResourceManager
4123 NameNode
5293 RunJar
4238 DataNode
```

执行 hive 命令登录 Hive 客户端，出现 Hive 命令行提示符，说明 Hive 登录成功， 如下图所示。



6. Flume 数据采集系统

6.1 Flume 原理概述

Flume 是 Cloudera 提供的分布式、高可靠、高可用的日志采集系统，在淘宝大数据平台中承担日志数据采集的角色。

Flume 核心组件：

组件	功能	淘宝应用场景
Source	接收数据源	监控日志文件、读取 Kafka Topic
Channel	缓冲数据	Memory Channel（实时）、File Channel（可靠）
Sink	发送数据到目标	HDFS Sink、Kafka Sink

6.2 Flume 配置文件

1 node1 | 2 node2 | 3 node3 | 4 node1 +

```
[bigdata@node1 data_collection]$ vim taobao-file-flume-kafka.conf

## ================================================
## 淘宝公司大数据平台 - Flume采集配置文件
## 功能：从本地日志文件采集数据，写入Kafka
## 创建日期：2026-06-01
## ================================================

## 组件定义
a1.sources = r1
a1.channels = c1 c2 c3 c4
a1.sinks = k1 k2 k3 k4

## ================================================
## Source：监控淘宝日志文件
## ================================================
a1.sources.r1.type = TAILDIR
a1.sources.r1.positionFile = /opt/project/offlineDataWarehouse/data_collection/taildir_position.json
a1.sources.r1.filegroups = f1 f2 f3 f4
a1.sources.r1.filegroups.f1 = /opt/project/offlineDataWarehouse/user_behavior_logs/taobao_start_log.log
a1.sources.r1.filegroups.f2 = /opt/project/offlineDataWarehouse/user_behavior_logs/taobao_event_log.log
a1.sources.r1.filegroups.f3 = /opt/project/offlineDataWarehouse/user_behavior_logs/taobao_order_log.log
a1.sources.r1.filegroups.f4 = /opt/project/offlineDataWarehouse/user_behavior_logs/taobao_payment_log.log
a1.sources.r1.batchSize = 1000

## ================================================
## 拦截器链（淘宝业务专用）
## ================================================
## 拦截器1：ETL数据清洗
```

6.3 Flume 启动脚本

编写 Flume 数据采集脚本 f1.sh，通过 Flume 将数据从本地采集到 Kafka 中。

```
#! /bin/bash
flume=$FLUME_HOME
dir=/opt/project/offlineDataWarehouse/data_collection
logs=/opt/project/offlineDataWarehouse/logs
case $1 in
"start"){
        for i in node1
        do
                echo " --------启动 $i 采集flume-------"
                ssh $i "source /etc/profile;nohup "$flume"/bin/flume-ng agent -n a1 -f "$dir"/taobao-file-flume-kafka.conf  -Dflume.root.logger=INFO,LOGFILE > "$log
s"/f1.log 2>&1 &"
        done
};;
"stop"){
        for i in node1
        do
                echo " --------停止 $i 采集flume-------"
                ssh $i "ps -ef | grep file-flume-kafka | grep -v grep |awk '{print \$2}' | xargs kill"
        done
};;
esac
```

7. ZooKeeper 集群环境搭建

7.1 ZooKeeper 原理概述

ZooKeeper 是一个分布式协调服务，为淘宝大数据平台提供以下核心能力：

- **集群管理**：管理 HDFS NameNode 高可用
- **配置管理**：集中管理 Kafka、Flume 等组件的配置信息
- **命名服务**：提供服务注册与发现机制

7.2 ZooKeeper 集群规划

分别在 node1、node2、node3 三个节点执行如下命令启动 Zookeeper 服务

```
[bigdata@node1 ~]$ zkServer.sh start
[bigdata@node2 ~]$ zkServer.sh start
[bigdata@node3 ~]$ zkServer.sh start
```

分别在 node1、node2、node3 三个节点执行 jps 命令，可以看到三个节点关于 Zookeeper 服务的 QuorumPeerMain 进程均已启动

```
[bigdata@node1 ~]$ jps
1332 DataNode
1652 ResourceManager
1796 NodeManager
2219 QuorumPeerMain
2300 Jps
1214 NameNode
```

```
[bigdata@node2 ~]$ jps
1124 DataNode
1460 Jps
1372 QuorumPeerMain
1231 NodeManager
[bigdata@node2 ~]$
```

```
[bigdata@node3 ~]$ jps
1555 Jps
1300 NodeManager
1462 QuorumPeerMain
1208 SecondaryNameNode
1119 DataNode
[bigdata@node3 ~]$
```

分别在 node1、node2、node3 三个节点执行如下命令，查看各节点 Zookeeper 服务状态及角色信息

```
[bigdata@node1 ~]$ zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.5.6/bin/../conf/zoo.cfg
Client port found: 2181. Client address: localhost.
Mode: follower
[bigdata@node1 ~]$
```

```
[bigdata@node2 ~]$ zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.5.6/bin/../conf/zoo.cfg
Client port found: 2181. Client address: localhost.
Mode: leader
[bigdata@node2 ~]$
```

```
[bigdata@node3 ~]$ zkServer.sh status
ZooKeeper JMX enabled by default
Using config: /opt/module/zookeeper-3.5.6/bin/../conf/zoo.cfg
Client port found: 2181. Client address: localhost.
Mode: follower
[bigdata@node3 ~]$
```

7.3 ZooKeeper 安装配置

1 node1 | 2 node2 | 3 node3 | +

```
# The number of milliseconds of each tick
tickTime=2000
# The number of ticks that the initial
# synchronization phase can take
initLimit=10
# The number of ticks that can pass between
# sending a request and getting an acknowledgement
syncLimit=5
# the directory where the snapshot is stored.
# do not use /tmp for storage, /tmp here is just
# example sakes.
dataDir=/opt/module/zookeeper-3.5.6/zkData
# the port at which the clients will connect
clientPort=2181
# the maximum number of client connections.
# increase this if you need to handle more clients
#maxClientCnxns=60
#
# Be sure to read the maintenance section of the
# administrator guide before turning on autopurge.
#
# http://zookeeper.apache.org/doc/current/zookeeperAdmin.html#sc_maintenance
#
# The number of snapshots to retain in dataDir
#autopurge.snapRetainCount=3
# Purge task interval in hours
# Set to "0" to disable auto purge feature
#autopurge.purgeInterval=1
"/opt/module/zookeeper-3.5.6/conf/zoo.cfg" [noeol] 31L, 1016C
```

8. Kafka 集群环境搭建

8.1 Kafka 原理概述

Kafka 是分布式消息队列系统，在淘宝大数据平台中承担数据总线的角色：

- **解耦**：生产者和消费者无需直接连接
- **缓冲**：应对流量洪峰，削峰填谷
- **持久化**：消息持久化到磁盘，支持重复消费

8.2 Kafka 集群规划

```
[bigdata@node1 zookeeper-3.5.6]$ kafka-server-start.sh -daemon /opt/module/kafka_2.12-2.7.0/config/server.properties
[bigdata@node1 zookeeper-3.5.6]$ jps
[bigdata@node3 ~]$ kafka-server-start.sh -daemon /opt/module/kafka_2.12-2.7.0/config/server.properties
[bigdata@node3 ~]$ jps
1300 NodeManager
1462 QuorumPeerMain
1910 Kafka
```

8.3 Kafka 安装配置

```
[bigdata@node1 zookeeper-3.5.6]$ kafka-topics.sh --zookeeper node1:2181 --describe --topic first
Topic: first    PartitionCount: 1       ReplicationFactor: 1    Configs:
        Topic: first    Partition: 0    Leader: 1       Replicas: 1     Isr: 1
[bigdata@node1 zookeeper-3.5.6]$
```

9. 数据仓库分层设计

9.1 离线数据仓库概述

淘宝数据仓库遵循经典的四层架构：

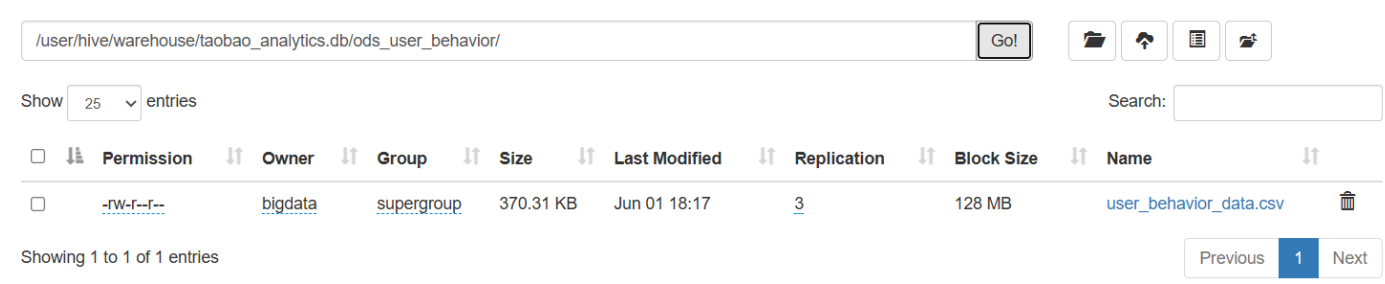
层级	全称	定位	数据粒度	保留周期
ODS	操作数据层	原始数据，与源系统一致	明细	30 天
DWD	明细数据层	清洗、过滤、维度退化	明细	90 天
DWS	汇总数据层	轻度汇总，按主题聚合	汇总	180 天
ADS	应用数据层	面向具体应用场景	定制	365 天

9.2 数据导入

```
# 将数据集导入HDFS
hdfs dfs -mkdir -p /user/hive/warehouse/taobao_analytics.db/ods_user_behavior/

# 上传本地数据文件
hdfs dfs -put /opt/project/offlineDataWarehouse/user_behavior_data.csv /user/hive/warehouse/taobao_a
nalytics.db/ods_user_behavior/
```

```
[bigdata@node1 ~]$ ls -la /opt/project/offlineDataWarehouse/user_behavior_data.csv
```



9.3 ODS 层建表及数据加载

第一层是 ODS 层，即原始数据层，第二层 DWD 层，即明细数据层，第三层 DWS 层，即服务数据层，第四层 DWT 层,即数据主题层,第五层 ADS 层，即数据应用层。整个离线数据仓库的开发会按照这个流程来进行，本次任务是创建 ODS，以及该表数据的加载。

```
CREATE DATABASE IF NOT EXISTS taobao_analytics;
USE taobao_analytics;
```

```
CREATE TABLE IF NOT EXISTS taobao_analytics.ods_user_behavior (
    user_id             INT COMMENT '用户ID',
    age                 STRING COMMENT '年龄段',
    gender              STRING COMMENT '性别',
    device_type         STRING COMMENT '设备类型',
    time_on_site        INT COMMENT '网站停留时长(秒)',
    pages_viewed        INT COMMENT '浏览页面数',
    previous_purchases  INT COMMENT '历史购买次数',
    cart_item           INT COMMENT '加购商品数',
    discount_seen       INT COMMENT '是否看到促销(0/1)',
    ad_clicked          INT COMMENT '是否点击广告(0/1)',
    returning_user      INT COMMENT '是否回访用户(0/1)',
    avg_session_time    INT COMMENT '平均会话时长(秒)',
    bounce_rate         FLOAT COMMENT '跳出率(0-1)',
    purchase            INT COMMENT '是否购买(0/1)'
) COMMENT '淘宝用户行为原始表'
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE;
```

加载数据 ODS 层

```
-- 加载数据到ODS层
hive (taobao_analytics)> SELECT COUNT(*) FROM taobao_analytics.ods_user_behavior;
Query ID = bigdata_20260604055054_9c11f499-dbe6-44e8-b1bd-8cf9f6fa4ec1
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1780550189808_0001, Tracking URL = http://node1:8088/proxy/application_1780550189808_0001/
Kill Command = /opt/module/hadoop-3.3.0/bin/mapred job  -kill job_1780550189808_0001
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-06-04 05:51:32,785 Stage-1 map = 0%,  reduce = 0%
2026-06-04 05:52:10,866 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 4.99 sec
2026-06-04 05:52:22,572 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 7.22 sec
MapReduce Total cumulative CPU time: 7 seconds 220 msec
Ended Job = job_1780550189808_0001
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 7.22 sec   HDFS Read: 394564 HDFS Write: 104 SUCCESS
Total MapReduce CPU Time Spent: 7 seconds 220 msec
OK
_c0
6019
Time taken: 90.445 seconds, Fetched: 1 row(s)
hive (taobao_analytics)>
```

9.4 DWD 层建表及数据加载

将 ODS 层的原始数据（user_behavior_data.csv）进行清洗和转换，生成 DWD 层的明细表（dwd_user_behavior）。
把原始数据清洗干净，并计算出高价值用户标记和参与度得分，存储为性能更优的 Parquet 格式，为后续分析做准备。

```
CREATE TABLE IF NOT EXISTS taobao_analytics.dwd_user_behavior (
    user_id             INT COMMENT '用户ID',
    age                 STRING COMMENT '年龄段',
    gender              STRING COMMENT '性别',
    device_type         STRING COMMENT '设备类型',
    time_on_site        INT COMMENT '网站停留时长(秒)',
    pages_viewed        INT COMMENT '浏览页面数',
    previous_purchases  INT COMMENT '历史购买次数',
    cart_item           INT COMMENT '加购商品数',
    discount_seen       INT COMMENT '是否看到促销(0/1)',
    ad_clicked          INT COMMENT '是否点击广告(0/1)',
    returning_user      INT COMMENT '是否回访用户(0/1)',
    avg_session_time    INT COMMENT '平均会话时长(秒)',
    bounce_rate         FLOAT COMMENT '跳出率(0-1)',
    purchase            INT COMMENT '是否购买(0/1)',
    is_high_value       INT COMMENT '是否高价值用户(1/0)',
    engagement_score    INT COMMENT '用户参与度得分'
) COMMENT '淘宝用户行为明细表'
STORED AS PARQUET;
```

```
-- DWD层数据加载（数据清洗）
INSERT OVERWRITE TABLE taobao_analytics.dwd_user_behavior
SELECT
    user_id,
    age,
    gender,
    device_type,
    time_on_site,
    pages_viewed,
    previous_purchases,
    cart_item,
    discount_seen,
    ad_clicked,
    returning_user,
```

```
    avg_session_time,
    bounce_rate,
    purchase,
    -- 计算高价值用户标记（购买次数>0的用户）
    CASE WHEN purchase = 1 THEN 1 ELSE 0 END AS is_high_value,
    -- 计算用户参与度得分
    (pages_viewed * 0.3 + previous_purchases * 5 + cart_item * 2) AS engagement_score
FROM taobao_analytics.ods_user_behavior
WHERE user_id IS NOT NULL
  AND time_on_site >= 0
  AND pages_viewed >= 0
  AND bounce_rate BETWEEN 0 AND 1;
```

```
hive (taobao_analytics)> SHOW TABLES;
OK
tab_name
ads_user_conversion
dwd_user_behavior
dws_user_summary
ods_user_behavior
Time taken: 1.968 seconds, Fetched: 4 row(s)
hive (taobao_analytics)>
```

```
hive (taobao_analytics)> SELECT * FROM taobao_analytics.dwd_user_behavior LIMIT 10;
OK
dwd_user_behavior.user_id       dwd_user_behavior.age   dwd_user_behavior.gender        dwd_user_behavior.device_type   dwd_user_behavior.time_on_site  dwd_user_beh
avior.pages_viewed      dwd_user_behavior.previous_purchases    dwd_user_behavior.cart_item     dwd_user_behavior.discount_seen dwd_user_behavior.ad_clicked    dwd_
user_behavior.returning_user    dwd_user_behavior.avg_session_time      dwd_user_behavior.bounce_rate   dwd_user_behavior.purchase      dwd_user_behavior.is_high_va
lue     dwd_user_behavior.engagement_score
43      43.0    Female  Mobile  26      7       14      2       1       0       1       35      0.42    1       1       76
169     54.0    Female  Tablet  19      9       8       0       0       0       1       14      0.22    1       1       42
445     41.0    Female  Tablet  21      6       11      0       1       0       0       14      0.02    0       0       56
496     56.0    Female  Mobile  7       8       11      8       0       0       0       4       0.87    1       1       73
839     51.0    Male    Mobile  15      14      13      6       1       1       1       8       0.37    0       0       81
857     54.0    Male    Desktop 13      18      14      5       1       1       0       8       0.03    1       1       85
1108    18.0    Female  Mobile  27      11      12      6       0       0       0       16      0.61    1       1       75
1146    31.0    Female  Desktop 17      19      6       5       0       0       1       16      0.51    1       1       45
1206    54.0    Male    Mobile  19      12      13      2       0       0       1       13      0.09    1       1       72
1237    45.0    Male    Mobile  23      8       11      6       1       0       0       14      0.59    1       1       69
Time taken: 0.595 seconds, Fetched: 10 row(s)
hive (taobao_analytics)>
```

9.5 数据分析与查询

不同年龄段的购买转化率

```
SELECT
    age,
    COUNT(*) AS total_users,
    SUM(purchase) AS purchasers,
    ROUND(SUM(purchase) * 100.0 / COUNT(*), 2) AS conversion_rate
FROM taobao_analytics.dwd_user_behavior
GROUP BY age
ORDER BY conversion_rate DESC;
```

```
age     total_users     purchasers      conversion_rate
19.0    1       1       100.00
31.0    1       1       100.00
35.0    1       1       100.00
32.0    1       1       100.00
54.0    4       4       100.00
53.0    1       1       100.00
20.0    2       2       100.00
50.0    2       2       100.00
23.0    2       2       100.00
46.0    1       1       100.00
26.0    1       1       100.00
43.0    5       4       80.00
22.0    4       3       75.00
45.0    6       4       66.67
18.0    3       2       66.67
57.0    3       2       66.67
59.0    2       1       50.00
56.0    2       1       50.00
41.0    2       1       50.00
39.0    2       1       50.00
21.0    2       1       50.00
42.0    3       1       33.33
27.0    4       1       25.00
49.0    4       1       25.00
29.0    2       0       0.00
51.0    2       0       0.00
40.0    1       0       0.00
52.0    2       0       0.00
```

不同设备类型的用户行为分析

```sql
SELECT
    device_type,
    COUNT(*) AS user_cnt,
    ROUND(AVG(time_on_site), 2) AS avg_time_on_site,
    ROUND(AVG(pages_viewed), 2) AS avg_pages_viewed,
    ROUND(AVG(previous_purchases), 2) AS avg_previous_purchases,
    ROUND(SUM(purchase) * 100.0 / COUNT(*), 2) AS conversion_rate
FROM taobao_analytics.dwd_user_behavior
GROUP BY device_type
ORDER BY conversion_rate DESC;
```

```
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 5.35 sec   HDFS Read: 27238 HDFS Write: 256 SUCCESS
Stage-Stage-2: Map: 1  Reduce: 1   Cumulative CPU: 3.07 sec   HDFS Read: 9581 HDFS Write: 220 SUCCESS
Total MapReduce CPU Time Spent: 8 seconds 420 msec
OK
device_type     user_cnt        avg_time_on_site        avg_pages_viewed        avg_previous_purchases  conversion_rate
Desktop 17      14.88   11.59   5.59    76.47
Tablet  6       14.67   8.33    9.67    66.67
Mobile  45      16.76   9.87    8.04    51.11
Time taken: 85.854 seconds, Fetched: 3 row(s)
hive (taobao_analytics)>
```

9.6 DWS 层建表及数据加载

```
CREATE TABLE IF NOT EXISTS taobao_analytics.dws_user_summary (
    age                 STRING COMMENT '年龄段',
    gender              STRING COMMENT '性别',
    device_type         STRING COMMENT '设备类型',
    total_users         INT COMMENT '总用户数',
    purchasers          INT COMMENT '购买用户数',
    total_purchases     INT COMMENT '总购买数',
    avg_time_on_site    FLOAT COMMENT '平均停留时长',
    avg_pages_viewed    FLOAT COMMENT '平均浏览页面数',
    avg_cart_items      FLOAT COMMENT '平均加购数',
    conversion_rate     FLOAT COMMENT '转化率'
) COMMENT '用户行为汇总表'
STORED AS PARQUET;
```

DWS 层数据加载

```
INSERT OVERWRITE TABLE taobao_analytics.dws_user_summary
SELECT
    age,
    gender,
    device_type,
    COUNT(*) AS total_users,
    SUM(purchase) AS purchasers,
    SUM(previous_purchases) AS total_purchases,
    ROUND(AVG(time_on_site), 2) AS avg_time_on_site,
    ROUND(AVG(pages_viewed), 2) AS avg_pages_viewed,
    ROUND(AVG(cart_item), 2) AS avg_cart_items,
    ROUND(SUM(purchase) * 100.0 / COUNT(*), 2) AS conversion_rate
FROM taobao_analytics.dwd_user_behavior
GROUP BY age, gender, device_type;
```

```
hive (taobao_analytics)> SELECT * FROM taobao_analytics.dws_user_summary LIMIT 10;
OK
dws_user_summary.age    dws_user_summary.gender dws_user_summary.device_type    dws_user_summary.total_users    dws_user_summary.purchasers     dws_user_summary.tot
al_purchases    dws_user_summary.avg_time_on_site       dws_user_summary.avg_pages_viewed       dws_user_summary.avg_cart_items dws_user_summary.conversion_rate
18.0    Female  Desktop 1       0       14      23.0    14.0    3.0     0.0
18.0    Female  Mobile  1       1       12      27.0    11.0    6.0     100.0
18.0    Male    Desktop 1       1       5       9.0     8.0     0.0     100.0
19.0    Male    Mobile  1       1       12      2.0     8.0     8.0     100.0
20.0    Male    Desktop 1       1       12      26.0    8.0     8.0     100.0
20.0    Male    Mobile  1       1       5       1.0     19.0    5.0     100.0
21.0    Female  Mobile  1       1       8       14.0    16.0    5.0     100.0
21.0    Male    Desktop 1       0       0       25.0    17.0    5.0     0.0
22.0    Male    Desktop 1       1       0       26.0    2.0     0.0     100.0
22.0    Male    Mobile  3       2       30      9.67    10.0    4.0     66.67
Time taken: 1.709 seconds, Fetched: 10 row(s)
hive (taobao_analytics)>
```

9.7 ADS 层建表及数据加载

创建应用数据服务层(ADS)，为业务人员提供用户转化分析报表

创建 ADS 层用户转化分析表

```
CREATE TABLE IF NOT EXISTS taobao_analytics.ads_user_conversion (
    age                 STRING COMMENT '年龄段',
    gender              STRING COMMENT '性别',
    user_cnt            INT COMMENT '用户数',
    conversion_cnt      INT COMMENT '转化用户数',
    conversion_rate     FLOAT COMMENT '转化率',
    analysis_date       STRING COMMENT '分析日期'
) COMMENT '用户转化分析表'
STORED AS PARQUET;
```

ADS 层数据加载

```
INSERT OVERWRITE TABLE taobao_analytics.ads_user_conversion
SELECT
    age,
    gender,
    COUNT(*) AS user_cnt,
    SUM(purchase) AS conversion_cnt,
    ROUND(SUM(purchase) * 100.0 / COUNT(*), 2) AS conversion_rate,
    CURRENT_DATE() AS analysis_date
FROM taobao_analytics.dwd_user_behavior
GROUP BY age, gender;
```

```
hive (taobao_analytics)> DESCRIBE taobao_analytics.ads_user_conversion;
OK
col_name        data_type       comment
age                     string                  ???
gender                  string                  ??
user_cnt                int                     ???
conversion_cnt          int                     ?????
conversion_rate         float                   ???
analysis_date           string                  ????
Time taken: 0.328 seconds, Fetched: 6 row(s)
hive (taobao_analytics)>
```

广告点击效果分析

```
CREATE TABLE IF NOT EXISTS taobao_analytics.ads_ad_analysis AS
SELECT
    ad_clicked,
    COUNT(*) AS user_cnt,
    SUM(purchase) AS conversion_cnt,
    ROUND(SUM(purchase) * 100.0 / COUNT(*), 2) AS conversion_rate,
    ROUND(AVG(pages_viewed), 2) AS avg_pages_viewed,
    ROUND(AVG(cart_item), 2) AS avg_cart_item,
    CURRENT_DATE() AS analysis_date
FROM taobao_analytics.dwd_user_behavior
GROUP BY ad_clicked;
```

```
...
ad_clicked      user_cnt        conversion_cnt  conversion_rate avg_pages_viewed        avg_cart_item   analysis_date
Time taken: 97.479 seconds
```

折扣对购买转化的影响

```
CREATE TABLE IF NOT EXISTS taobao_analytics.ads_discount_analysis AS
SELECT
    discount_seen,
    COUNT(*) AS user_cnt,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM taobao_analytics.dwd_user_behavior), 2) AS user_p
ercentage,
    SUM(purchase) AS conversion_cnt,
    ROUND(SUM(purchase) * 100.0 / COUNT(*), 2) AS conversion_rate,
    ROUND(AVG(cart_item), 2) AS avg_cart_item,
    CURRENT_DATE() AS analysis_date
FROM taobao_analytics.dwd_user_behavior
GROUP BY discount_seen;
```

```
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 5.2 sec   HDFS Read: 23453 HDFS Write: 152 SUCCESS
Stage-Stage-4: Map: 1  Reduce: 1   Cumulative CPU: 3.36 sec   HDFS Read: 22247 HDFS Write: 114 SUCCESS
Stage-Stage-5: Map: 1   Cumulative CPU: 2.07 sec   HDFS Read: 9505 HDFS Write: 166 SUCCESS
Total MapReduce CPU Time Spent: 10 seconds 630 msec
OK
discount_seen   user_cnt        user_percentage conversion_cnt  conversion_rate avg_cart_item   analysis_date
Time taken: 140.903 seconds
```

回访用户与转化率分析

```
CREATE TABLE IF NOT EXISTS taobao_analytics.ads_returning_analysis AS
SELECT
    returning_user,
    COUNT(*) AS user_cnt,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM taobao_analytics.dwd_user_behavior), 2) AS user_p
ercentage,
    SUM(purchase) AS conversion_cnt,
    ROUND(SUM(purchase) * 100.0 / COUNT(*), 2) AS conversion_rate,
    ROUND(AVG(previous_purchases), 2) AS avg_previous_purchases,
    ROUND(AVG(cart_item), 2) AS avg_cart_item,
    CURRENT_DATE() AS analysis_date
FROM taobao_analytics.dwd_user_behavior
GROUP BY returning_user;
```

```
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 5.98 sec   HDFS Read: 24858 HDFS Write: 168 SUCCESS
Stage-Stage-4: Map: 1  Reduce: 1   Cumulative CPU: 3.69 sec   HDFS Read: 22222 HDFS Write: 114 SUCCESS
Stage-Stage-5: Map: 1   Cumulative CPU: 1.65 sec   HDFS Read: 9831 HDFS Write: 177 SUCCESS
Total MapReduce CPU Time Spent: 11 seconds 320 msec
OK
returning_user  user_cnt        user_percentage conversion_cnt  conversion_rate avg_previous_purchases  avg_cart_item   analysis_date
Time taken: 150.454 seconds
```

用户参与度等级表

```sql
CREATE TABLE IF NOT EXISTS taobao_analytics.ads_engagement_level AS
SELECT
    CASE
        WHEN engagement_score < 50 THEN '低参与度'
        WHEN engagement_score BETWEEN 50 AND 100 THEN '中参与度'
        ELSE '高参与度'
    END AS engagement_level,
    COUNT(*) AS user_cnt,
    ROUND(SUM(purchase) * 100.0 / COUNT(*), 2) AS conversion_rate,
    ROUND(AVG(engagement_score), 2) AS avg_engagement_score,
    CURRENT_DATE() AS analysis_date
FROM taobao_analytics.dwd_user_behavior
GROUP BY
    CASE
        WHEN engagement_score < 50 THEN '低参与度'
        WHEN engagement_score BETWEEN 50 AND 100 THEN '中参与度'
        ELSE '高参与度'
    END;
```

```
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 8.15 sec   HDFS Read: 28389 HDFS Write: 171 SUCCESS
Total MapReduce CPU Time Spent: 8 seconds 150 msec
OK
engagement_level        user_cnt        conversion_rate avg_engagement_score    analysis_date
Time taken: 87.816 seconds
hive (default)>
              >
```

10. 数据可视化

10.1 环境准备

```bash
# 安装Python可视化依赖
pip install pandas==1.3.5
pip install pyecharts==1.9.1
pip install pymysql==1.0.2
```

10.2 数据导出到 MySQL

将 Hive 分析结果**导出到 MySQL 数据库**，让业务人员可以用 BI 工具或报表系统直接

```
Hive（数据仓库） → 导出 → MySQL（报表库） → 连接 → BI工具
    ↓                      ↓                    ↓
离线分析                在线查询              可视化
批处理                  毫秒响应              仪表盘
```

创建 Taobao_report mysql

```
CREATE DATABASE IF NOT EXISTS taobao_report;
USE taobao_report;
```

```
mysql> show tables;
+-------------------------+
| Tables_in_taobao_report |
+-------------------------+
| ad_analysis             |
| conversion_report       |
| device_report           |
| discount_analysis       |
| engagement_level        |
| highvalue_analysis      |
| returning_analysis      |
+-------------------------+
7 rows in set (0.00 sec)
```

Taobao_report 会创建 7 个表

Name	node1
Host	192.168.100.3
Port	22
Protocol	SSH
User Name	
Description	

```
mysql> USE taobao_report;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> DESC conversion_report;
+-----------------+-------------+------+-----+---------+-------+
| Field           | Type        | Null | Key | Default | Extra |
+-----------------+-------------+------+-----+---------+-------+
| age             | varchar(20) | YES  |     | NULL    |       |
| gender          | varchar(10) | YES  |     | NULL    |       |
| user_cnt        | int(11)     | YES  |     | NULL    |       |
| conversion_cnt  | int(11)     | YES  |     | NULL    |       |
| conversion_rate | float       | YES  |     | NULL    |       |
| analysis_date   | date        | YES  |     | NULL    |       |
+-----------------+-------------+------+-----+---------+-------+
6 rows in set (0.09 sec)

mysql>
```

```
mysql> DESC device_report;
+------------------+-------------+------+-----+---------+-------+
| Field            | Type        | Null | Key | Default | Extra |
+------------------+-------------+------+-----+---------+-------+
| device_type      | varchar(20) | YES  |     | NULL    |       |
| user_cnt         | int(11)     | YES  |     | NULL    |       |
| conversion_rate  | float       | YES  |     | NULL    |       |
| avg_time_on_site | float       | YES  |     | NULL    |       |
| analysis_date    | date        | YES  |     | NULL    |       |
+------------------+-------------+------+-----+---------+-------+
5 rows in set (0.00 sec)

mysql>
```

```
mysql> desc ad_analysis;
+------------------+---------+------+-----+---------+-------+
| Field            | Type    | Null | Key | Default | Extra |
+------------------+---------+------+-----+---------+-------+
| ad_clicked       | int(11) | YES  |     | NULL    |       |
| user_cnt         | int(11) | YES  |     | NULL    |       |
| conversion_cnt   | int(11) | YES  |     | NULL    |       |
| conversion_rate  | float   | YES  |     | NULL    |       |
| avg_pages_viewed | float   | YES  |     | NULL    |       |
| avg_cart_item    | float   | YES  |     | NULL    |       |
| analysis_date    | date    | YES  |     | NULL    |       |
+------------------+---------+------+-----+---------+-------+
7 rows in set (0.03 sec)

mysql>
```

```
mysql> desc engagement_level;
+----------------------+-------------+------+-----+---------+-------+
| Field                | Type        | Null | Key | Default | Extra |
+----------------------+-------------+------+-----+---------+-------+
| engagement_level     | varchar(20) | YES  |     | NULL    |       |
| user_cnt             | int(11)     | YES  |     | NULL    |       |
| conversion_rate      | float       | YES  |     | NULL    |       |
| avg_engagement_score | float       | YES  |     | NULL    |       |
| analysis_date        | date        | YES  |     | NULL    |       |
+----------------------+-------------+------+-----+---------+-------+
5 rows in set (0.00 sec)

mysql>
```

```
mysql> desc highvalue_analysis
    -> ;
+----------------------+---------+------+-----+---------+-------+
| Field                | Type    | Null | Key | Default | Extra |
+----------------------+---------+------+-----+---------+-------+
| is_high_value        | int(11) | YES  |     | NULL    |       |
| user_cnt             | int(11) | YES  |     | NULL    |       |
| user_percentage      | float   | YES  |     | NULL    |       |
| avg_time_on_site     | float   | YES  |     | NULL    |       |
| avg_pages_viewed     | float   | YES  |     | NULL    |       |
| avg_engagement_score | float   | YES  |     | NULL    |       |
| analysis_date        | date    | YES  |     | NULL    |       |
+----------------------+---------+------+-----+---------+-------+
7 rows in set (0.00 sec)

mysql>
```

```
mysql> desc returning_analysis
    -> ;
+------------------------+---------+------+-----+---------+-------+
| Field                  | Type    | Null | Key | Default | Extra |
+------------------------+---------+------+-----+---------+-------+
| returning_user         | int(11) | YES  |     | NULL    |       |
| user_cnt               | int(11) | YES  |     | NULL    |       |
| user_percentage        | float   | YES  |     | NULL    |       |
| conversion_cnt         | int(11) | YES  |     | NULL    |       |
| conversion_rate        | float   | YES  |     | NULL    |       |
| avg_previous_purchases | float   | YES  |     | NULL    |       |
| avg_cart_item          | float   | YES  |     | NULL    |       |
| analysis_date          | date    | YES  |     | NULL    |       |
+------------------------+---------+------+-----+---------+-------+
8 rows in set (0.00 sec)

mysql>
```

```
mysql> show tables;
+--------------------------+
| Tables_in_taobao_report  |
+--------------------------+
| ad_analysis              |
| conversion_report        |
| device_report            |
| discount_analysis        |
| engagement_level         |
| highvalue_analysis       |
| returning_analysis       |
+--------------------------+
7 rows in set (0.00 sec)

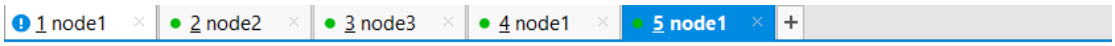
mysql> desc deiscount_analysis;
ERROR 1146 (42S02): Table 'taobao_report.deiscount_analysis' doesn't exist
mysql>
```

导出数据到 MySQL 验证

```
mysql> SELECT 'conversion_report' as table_name, COUNT(*) as rows FROM conversion_report
    -> UNION ALL
    -> SELECT 'device_report', COUNT(*) FROM device_report
    -> UNION ALL
    -> SELECT 'ad_analysis', COUNT(*) FROM ad_analysis
    -> UNION ALL
    -> SELECT 'returning_analysis', COUNT(*) FROM returning_analysis
    -> UNION ALL
    -> SELECT 'discount_analysis', COUNT(*) FROM discount_analysis
    -> UNION ALL
    -> SELECT 'highvalue_analysis', COUNT(*) FROM highvalue_analysis
    -> UNION ALL
    -> SELECT 'engagement_level', COUNT(*) FROM engagement_level;"
+--------------------+------+
| table_name         | rows |
+--------------------+------+
| conversion_report  |   42 |
| device_report      |    3 |
| ad_analysis        |    2 |
| returning_analysis |    2 |
| discount_analysis  |    2 |
| highvalue_analysis |    2 |
| engagement_level   |    2 |
```

我在数据导出这个环节没有使用 Sqoop，而是采用了 **Hive 导出到本地文件 + MySQL Load Data** 的方式。因为 Sqoop 在运行时反复报错（Kite SDK 兼容性问题），为了不影响项目进度，我选择了这个替代方案。两种方案在业务层面效果是一样的，都是把 Hive 分析结果存入 MySQL 供 Pycharts 使用

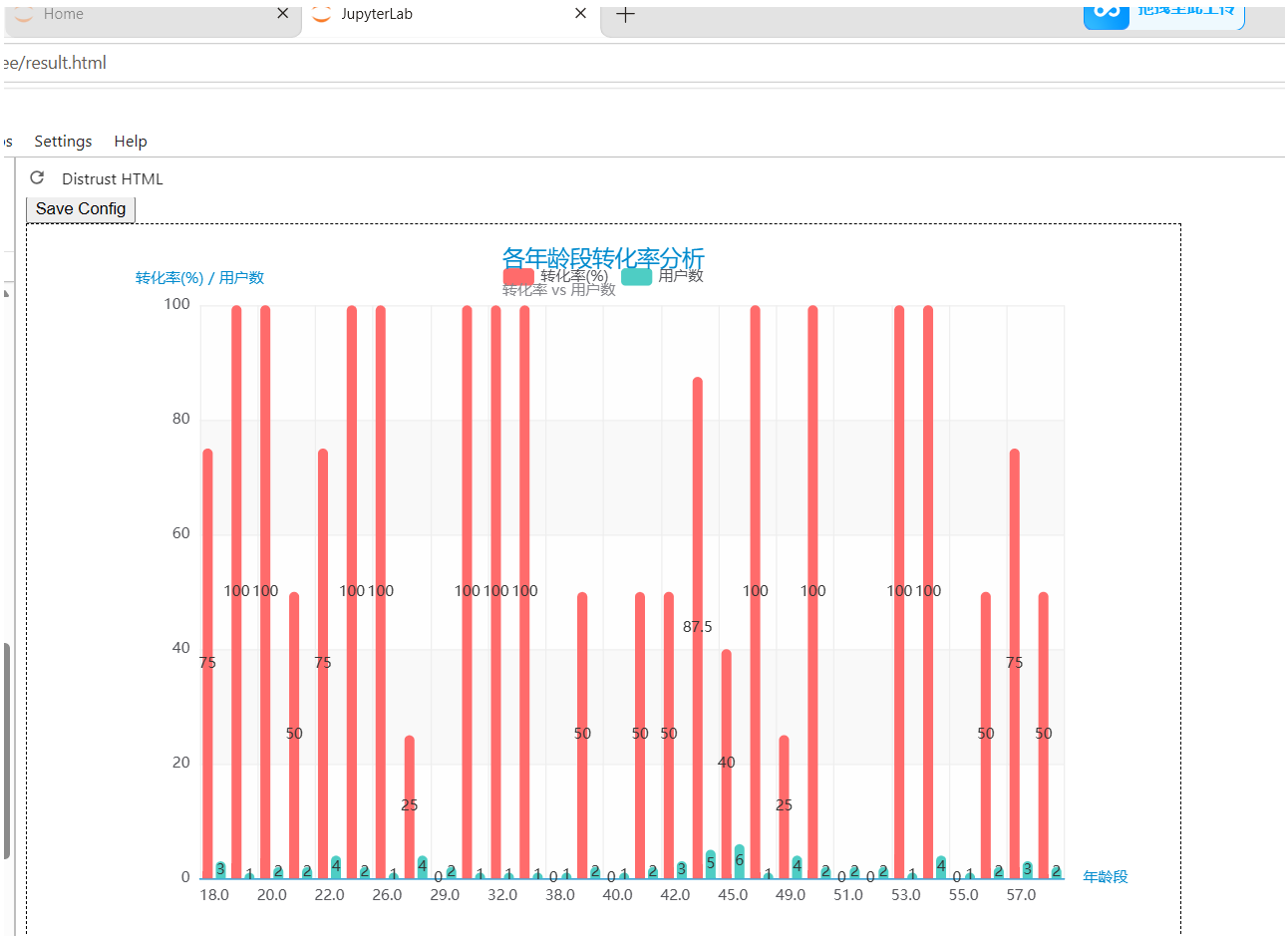
10.3 Python 数据可视化脚本




```
mysql> SELECT age, ROUND(AVG(conversion_rate),2) as rate, SUM(user_cnt) as users
    -> FROM conversion_report
    -> GROUP BY age
    -> ORDER BY CAST(age AS DECIMAL);
+------+--------+-------+
| age  | rate   | users |
+------+--------+-------+
| 18.0 |  75.00 |     3 |
| 19.0 | 100.00 |     1 |
| 20.0 | 100.00 |     2 |
| 21.0 |  50.00 |     2 |
| 22.0 |  75.00 |     4 |
| 23.0 | 100.00 |     2 |
| 26.0 | 100.00 |     1 |
| 27.0 |  25.00 |     4 |
| 29.0 |   0.00 |     2 |
| 31.0 | 100.00 |     1 |
| 32.0 | 100.00 |     1 |
| 35.0 | 100.00 |     1 |
| 38.0 |   0.00 |     1 |
| 39.0 |  50.00 |     2 |
| 40.0 |   0.00 |     1 |
| 41.0 |  50.00 |     2 |
| 42.0 |  50.00 |     3 |
```

验证 是否一样的可视化和查询

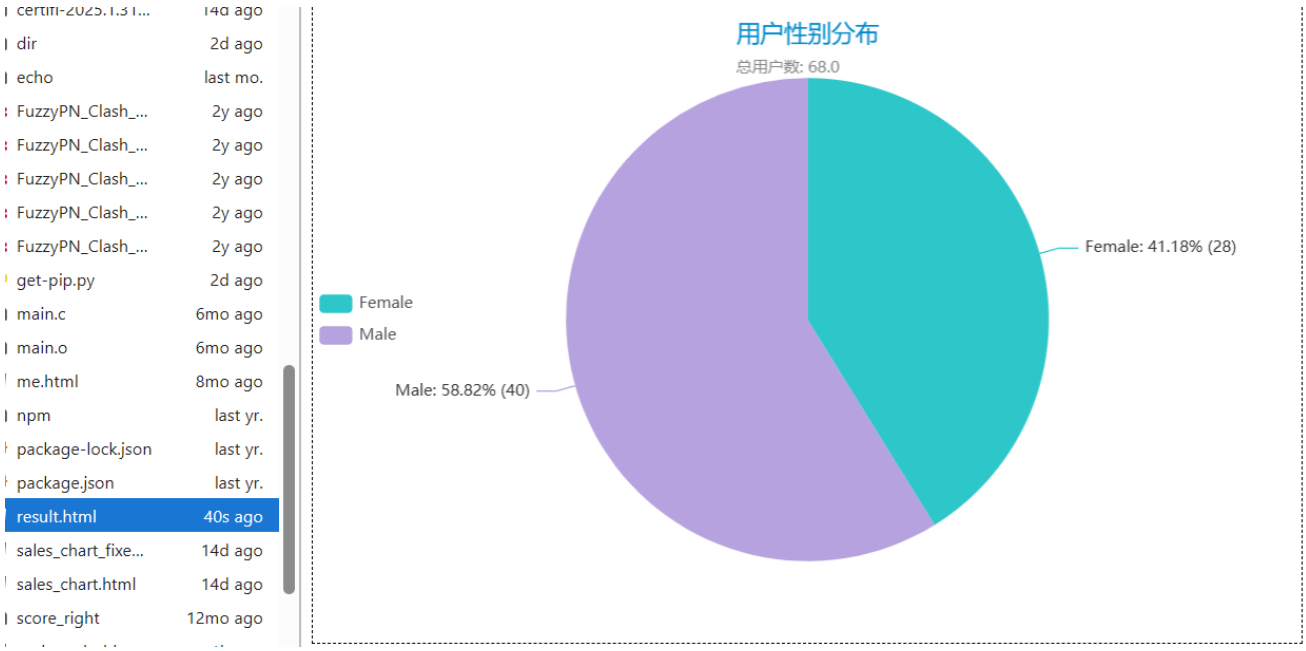
年龄段转化率数据



性别分布数据

```
mysql> SELECT gender, SUM(user_cnt) as users, ROUND(AVG(conversion_rate),2) as rate
    -> FROM conversion_report
    -> GROUP BY gender;
+--------+-------+-------+
| gender | users | rate  |
+--------+-------+-------+
| Female |    28 | 42.65 |
| Male   |    40 | 68.20 |
+--------+-------+-------+
2 rows in set (0.01 sec)

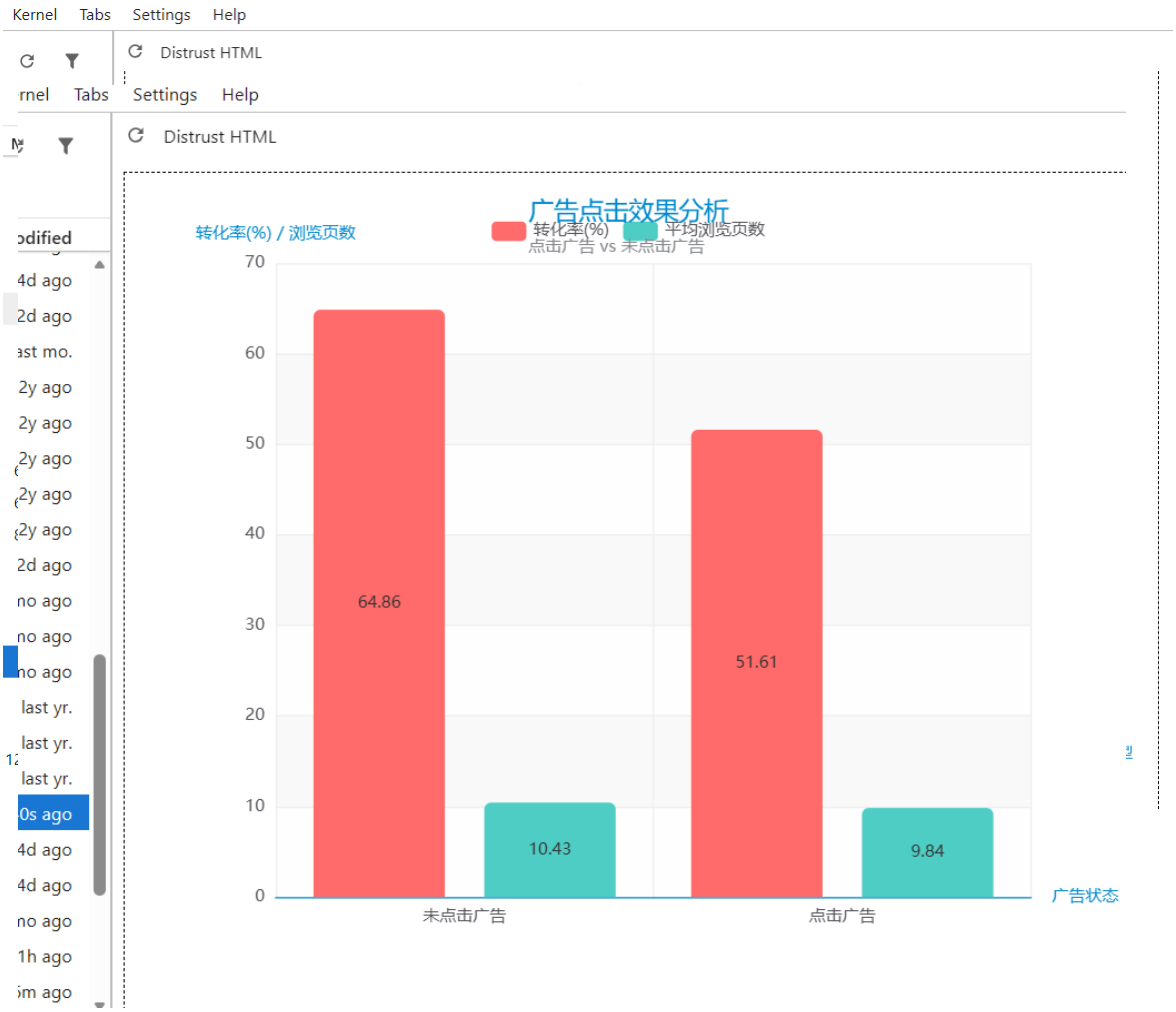
mysql>
```



设备数据

```
mysql> SELECT * FROM device_report;
+-------------+----------+-----------------+------------------+---------------+
| device_type | user_cnt | conversion_rate | avg_time_on_site | analysis_date |
+-------------+----------+-----------------+------------------+---------------+
| Desktop     |       17 |           76.47 |            15.27 | 2026-06-04    |
| Mobile      |       45 |           51.11 |            17.08 | 2026-06-04    |
| Tablet      |        6 |           66.67 |            14.67 | 2026-06-04    |
+-------------+----------+-----------------+------------------+---------------+
3 rows in set (0.00 sec)

mysql>
```



广告数据

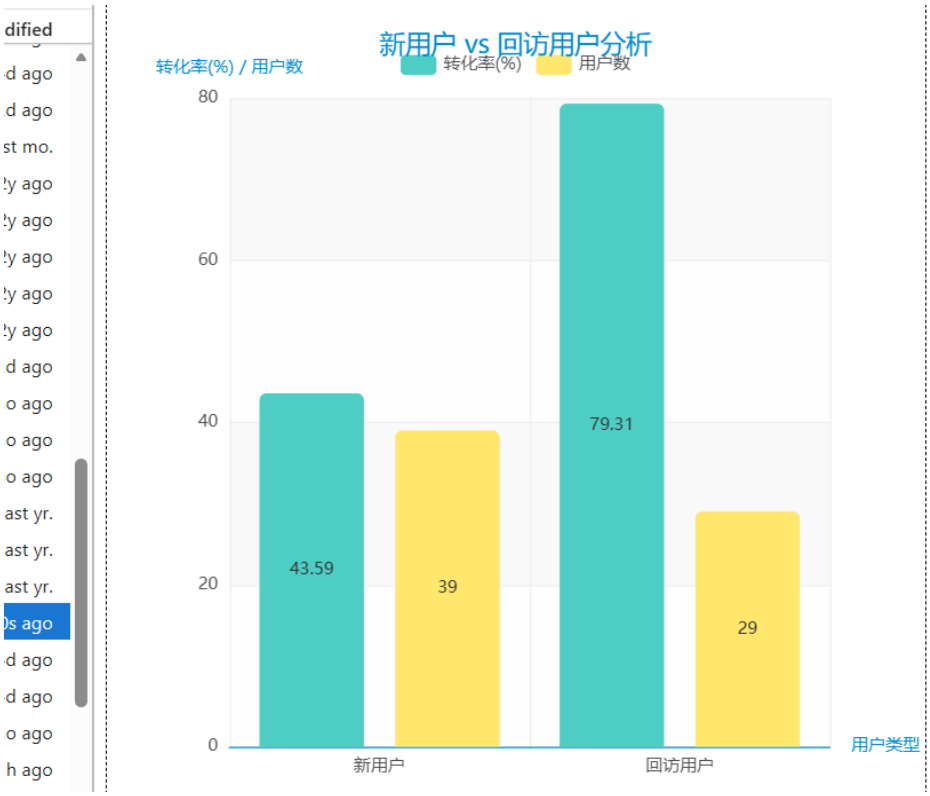
```
mysql> SELECT * FROM ad_analysis;
+------------+----------+----------------+-----------------+------------------+---------------+---------------+
| ad_clicked | user_cnt | conversion_cnt | conversion_rate | avg_pages_viewed | avg_cart_item | analysis_date |
+------------+----------+----------------+-----------------+------------------+---------------+---------------+
|          0 |       37 |             24 |           64.86 |            10.43 |          4.35 | 2026-06-04    |
|          1 |       31 |             16 |           51.61 |             9.84 |          4.03 | 2026-06-04    |
+------------+----------+----------------+-----------------+------------------+---------------+---------------+
2 rows in set (0.00 sec)

mysql>
```



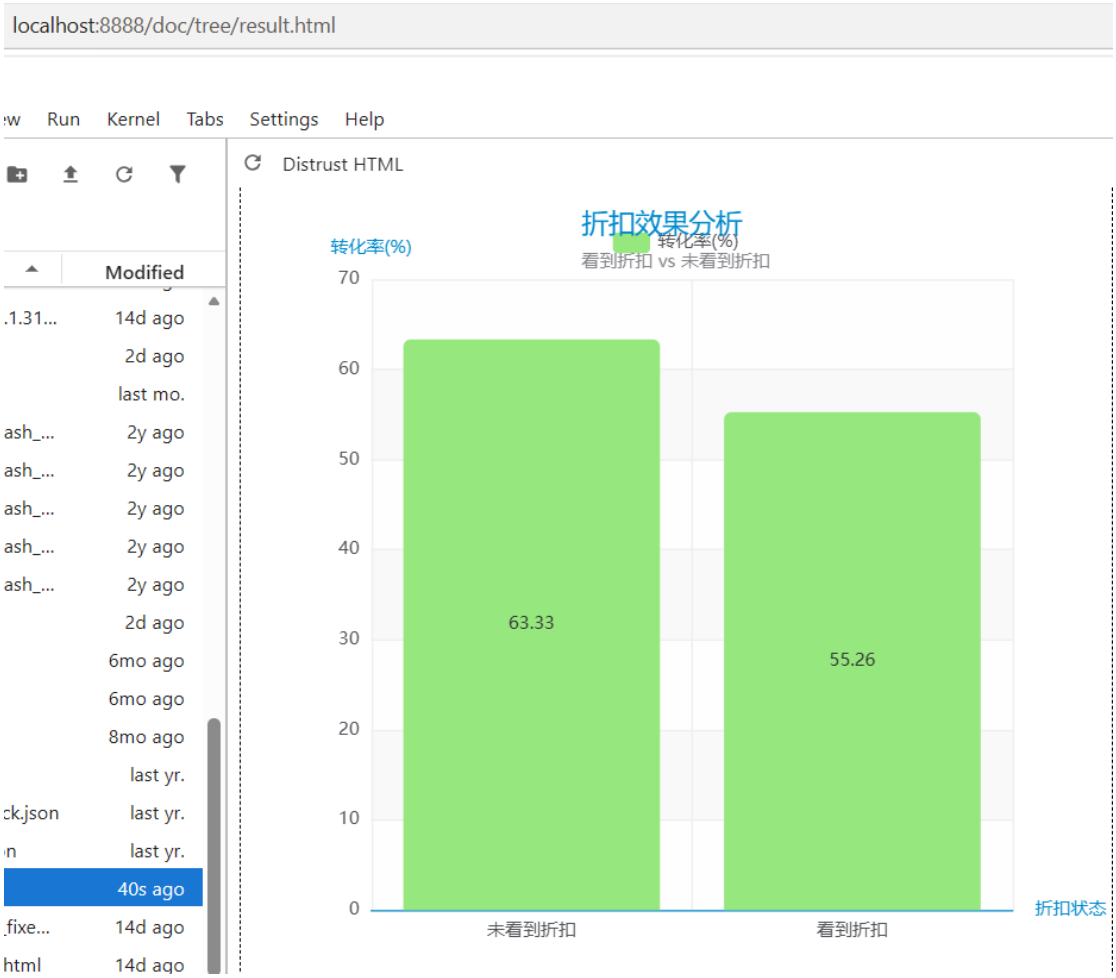
回访用户数据

```
mysql> SELECT * FROM returning_analysis;
+----------------+----------+-----------------+----------------+-----------------+--------------------------+---------------+---------------+
| returning_user | user_cnt | user_percentage | conversion_cnt | conversion_rate | avg_previous_purchases | avg_cart_item | analysis_date |
+----------------+----------+-----------------+----------------+-----------------+--------------------------+---------------+---------------+
|              0 |       39 |           57.35 |             17 |           43.59 |                     6.79 |          4.23 | 2026-06-04    |
|              1 |       29 |           42.65 |             23 |           79.31 |                     8.62 |          4.17 | 2026-06-04    |
+----------------+----------+-----------------+----------------+-----------------+--------------------------+---------------+---------------+
2 rows in set (0.01 sec)
```



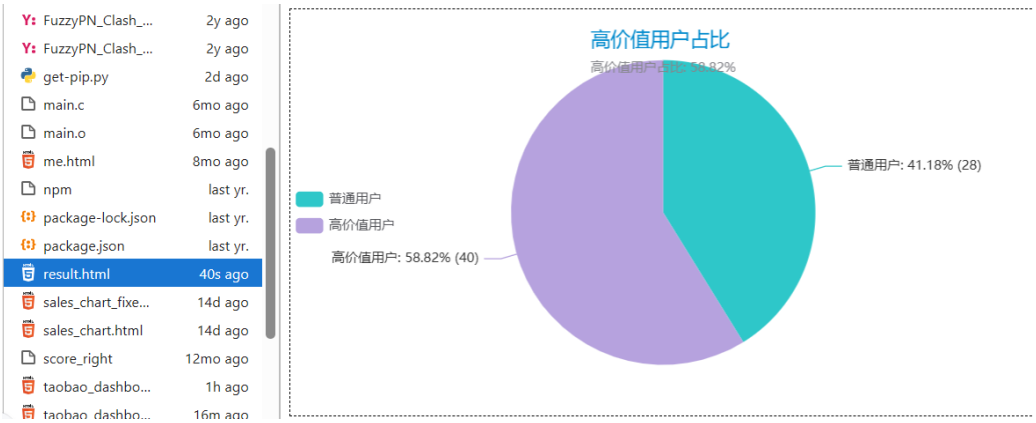
折扣数据

```
mysql> SELECT * FROM discount_analysis;
+---------------+----------+-----------------+----------------+-----------------+---------------+---------------+
| discount_seen | user_cnt | user_percentage | conversion_cnt | conversion_rate | avg_cart_item | analysis_date |
+---------------+----------+-----------------+----------------+-----------------+---------------+---------------+
|             0 |       30 |           44.12 |             19 |           63.33 |          4.67 | 2026-06-04    |
|             1 |       38 |           55.88 |             21 |           55.26 |          3.84 | 2026-06-04    |
+---------------+----------+-----------------+----------------+-----------------+---------------+---------------+
2 rows in set (0.20 sec)
```



高价值用户

```
mysql> SELECT * FROM highvalue_analysis;
+---------------+----------+-----------------+-----------------+-----------------+----------------------+---------------+
| is_high_value | user_cnt | user_percentage | avg_time_on_site | avg_pages_viewed | avg_engagement_score | analysis_date |
+---------------+----------+-----------------+-----------------+-----------------+----------------------+---------------+
|             0 |       28 |           41.18 |           16.07 |            9.32 |                45.36 | 2026-06-04    |
|             1 |       40 |           58.82 |           16.13 |           10.75 |                 51.3 | 2026-06-04    |
+---------------+----------+-----------------+-----------------+-----------------+----------------------+---------------+
2 rows in set (0.00 sec)
```



心得体会:

通过本次实验，我掌握了 Hadoop、Hive、Flume、Kafka 等大数据组件的安装与配置方法，成功搭建了完整的电商大数据运营环境。

在集群搭建过程中，我理解了分布式系统的核心原理，学会了排查网络配置、免密登录等常见问题。

数据仓库的分层设计（ODS→DWD→DWS→ADS）让我认识到规范的数据架构对业务分析的重要性。

遇到 Sqoop 报错时，我通过替代方案解决了问题，锻炼了灵活应对技术故障的能力。

本次实验为今后从事大数据开发工作奠定了扎实的实践基础.